Практическое занятие № 8. Работа с MongoDB: подключение, создание коллекции, CRUD-операции

Дисциплина: Технологии индустриального программирования
Тема: Работа с MongoDB из приложения на Go
Цель работы: изучить документную модель MongoDB, научиться подключаться к базе данных из Go, создавать коллекции и индексы, реализовывать CRUD-операции, фильтрацию, пагинацию, частичное обновление и обработку ошибок.

1. Теоретические сведения

MongoDB - документно-ориентированная система управления базами данных, относящаяся к классу NoSQL. В отличие от реляционных СУБД, в которых информация представлена таблицами со строками и столбцами, MongoDB хранит данные в виде документов, объединённых в коллекции.

Документ - самостоятельная запись в базе данных. По структуре он похож на JSON-объект, но физически MongoDB использует BSON (Binary JSON). BSON поддерживает типы данных, которые не представлены в обычном JSON, например ObjectID и дату. Внутри документа могут находиться вложенные объекты и массивы.

Коллекция - набор документов. Она функционально напоминает таблицу реляционной базы данных, однако не требует, чтобы все документы имели строго одинаковый набор полей. База данных объединяет несколько коллекций.

У каждого документа есть поле `_id`. Если оно не задано при добавлении документа, MongoDB создаёт его автоматически. В данной работе используется `ObjectID` - специальный идентификатор MongoDB. В приложении на Go для него применяется тип `primitive.ObjectID`, а для преобразования строкового представления в идентификатор используется `primitive.ObjectIDFromHex`.

Основные операции CRUD:

- **Create** - создание документа (`InsertOne`);
- **Read** - чтение одного документа или набора документов (`FindOne` , `Find`);
- **Update** - обновление документа (`UpdateOne` , `FindOneAndUpdate`), в том числе отдельных полей с помощью `$set` ;
- **Delete** - удаление документа (`DeleteOne`).

Индекс - вспомогательная структура, ускоряющая поиск и позволяющая накладывать ограничения. В этой работе создаётся уникальный индекс по полю `title`: он запрещает хранить две заметки с одинаковым заголовком. При попытке вставить дубликат MongoDB возвращает ошибку нарушения уникальности, а HTTP API преобразует её в ответ `409 Conflict`.

При обращении к MongoDB используются объекты `context.Context` с ограниченным временем выполнения. Контекст позволяет прекратить операцию, если база данных не отвечает в течение установленного срока, и не оставлять запросы приложения бесконечно ожидающими.

2. Подготовка программного окружения

2.1. Установка и проверка Go

Необходимо установить Go версии 1.21 или новее. После установки открыть PowerShell и выполнить:

```
go version
```

В результате должна отобразиться установленная версия Go. Если команда не распознаётся, следует проверить установку Go и наличие каталога Go в системной переменной `PATH`.

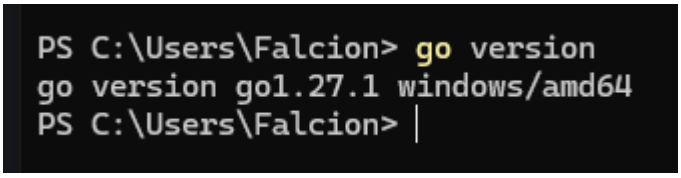


Рисунок 1 — Проверка установленной версии Go в PowerShell.

2.2. Установка Docker Desktop

Для запуска MongoDB используется Docker Compose. Необходимо установить Docker Desktop для Windows, запустить приложение и дождаться состояния, при котором Docker Engine работает.

Проверить доступность Docker можно командами:

```
docker --version
docker compose version
```

Обе команды должны вывести версии соответствующих компонентов. Если Docker не запускается, следует проверить, включена ли виртуализация в BIOS/UEFI и настроена ли поддерживаемая среда WSL 2, если Docker Desktop использует её в качестве backend.



Рисунок 2 — Проверка Docker и Docker Compose в PowerShell.

3. Создание проекта

3.1. Создание каталогов

В PowerShell перейдём в каталог, в котором будут храниться учебные проекты. Например, для рабочего стола:

```
Set-Location "$HOME\Desktop"
New-Item -ItemType Directory -Name pz8-mongo
Set-Location .\pz8-mongo
New-Item -ItemType Directory -Force -Path .\cmd\api, .\internal\db,
.\internal\notes
```

В результате создаётся структура каталогов, разделяющая точку входа приложения, работу с подключением к базе данных и функциональность заметок.

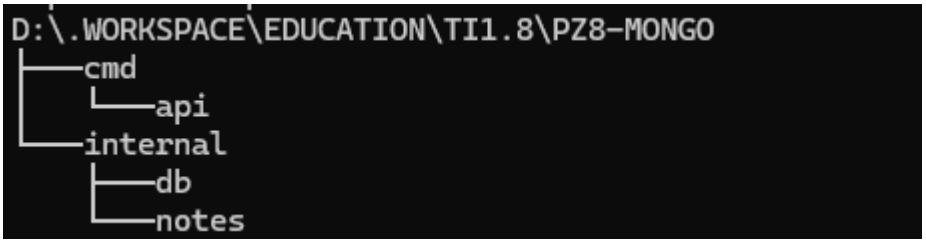


Рисунок 3 — Создание структуры каталогов проекта в PowerShell или её отображение в редакторе.

3.2. Инициализация Go-модуля

В корне проекта выполняется:

```
go mod init example.com/pz8-mongo
```

Команда создаёт файл `go.mod`, содержащий имя модуля и сведения о версии Go. В дальнейшем импорты внутренних пакетов будут начинаться с `example.com/pz8-mongo`.

Добавим зависимости проекта:

```
go get github.com/go-chi/chi/v5
go get github.com/joho/godotenv
go get go.mongodb.org/mongo-driver@v1.17.6
```

`chi` используется для маршрутизации HTTP-запросов, `godotenv` — для чтения настроек из файла `.env`, а официальный драйвер MongoDB обеспечивает взаимодействие приложения с базой данных.

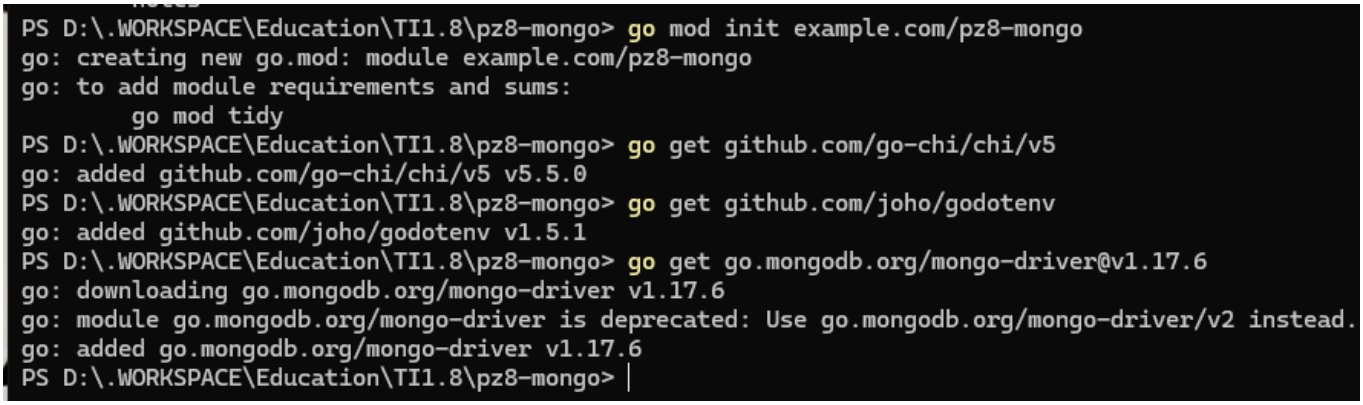


Рисунок 4 — Инициализация Go-модуля и установка зависимостей.

3.3. Конфигурация подключения

В корне проекта создаётся файл `.env.example` со следующим содержимым:

```
MONGO_URI=mongodb://root:secret@localhost:27017/?authSource=admin
MONGO_DB=pz8
HTTP_ADDR=:8080
```

Переменные имеют следующее назначение:

- `MONGO_URI` — адрес MongoDB и параметры аутентификации;
- `MONGO_DB` — имя базы данных, используемой приложением;
- `HTTP_ADDR` — адрес и порт HTTP-сервера.

Создадим рабочую копию файла:

```
Copy-Item .env.example .env
```

Файл `.env` содержит локальные настройки и не должен попадать в систему контроля версий. Поэтому он добавлен в `.gitignore`.

В строке подключения параметр `authSource=admin` указывает, что учётная запись `root` создана в административной базе данных MongoDB.

```
go: added go.mongodb.org/mongo-driver v1.17.8
PS D:\.WORKSPACE\Education\TI1.8\pz8-mongo> Copy-Item .env.example .env
PS D:\.WORKSPACE\Education\TI1.8\pz8-mongo> Get-Content .env
MONGO_URI=mongodb://root:secret@localhost:27017/?authSource=admin
MONGO_DB=pz8
HTTP_ADDR=:8080
PS D:\.WORKSPACE\Education\TI1.8\pz8-mongo>
```

Рисунок 5 — Файл `.env.example` и созданная копия `.env` с параметрами подключения.

4. Запуск MongoDB в Docker

В корне проекта создаётся `docker-compose.yml`:

```yaml
services:
  mongo:
    image: mongo:7
    container_name: mongo-dev
    restart: unless-stopped
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: secret
    volumes:
      - mongo_data:/data/db

volumes:
  mongo_data:
```

Файл описывает сервис MongoDB версии 7. Порт `27017` контейнера публикуется на таком же порту компьютера. Переменные окружения задают учётные данные администратора. Именованный том `mongo_data` сохраняет данные между перезапусками контейнера.

Запустим контейнер:

```
docker compose up -d
```

Ключ `-d` запускает контейнер в фоновом режиме. Проверим его состояние:

```
docker compose ps
```

В списке должен присутствовать контейнер `mongo-dev` со статусом `Up` или `running`, а порт `27017` должен быть опубликован.

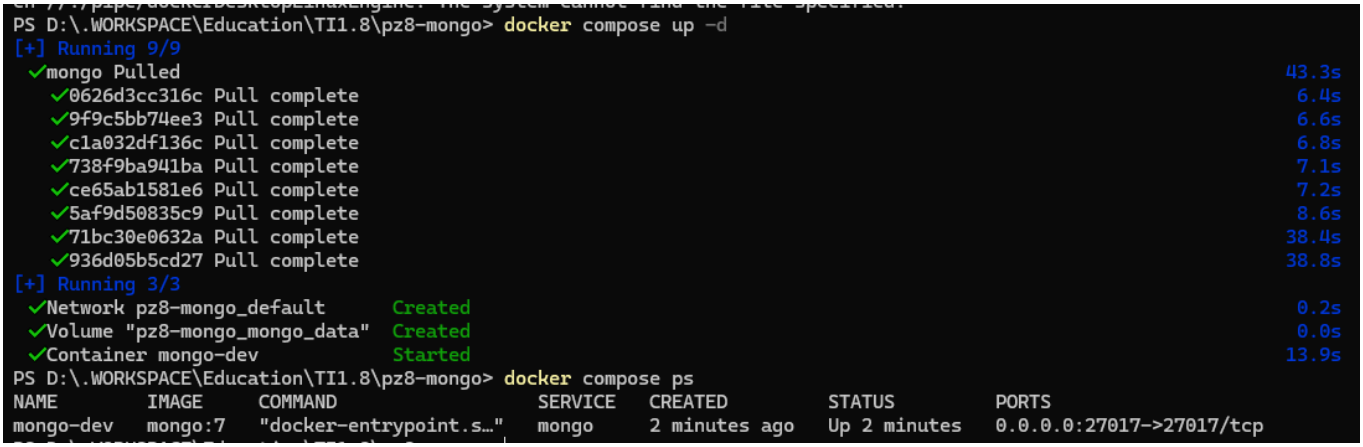


Рисунок 6 — Запуск MongoDB и проверка состояния контейнера командой `docker compose ps`.

Для дополнительной проверки можно открыть оболочку MongoDB:

```
docker exec -it mongo-dev mongosh -u root -p secret --authenticationDatabase admin
```

После подключения выполнить команды оболочки:

```
show dbs
exit
```

Если оболочка открылась и команда `show dbs` выполнилась, сервер MongoDB доступен. База `pz8` может пока отсутствовать в списке, поскольку MongoDB обычно создаёт её физически после первой записи данных.

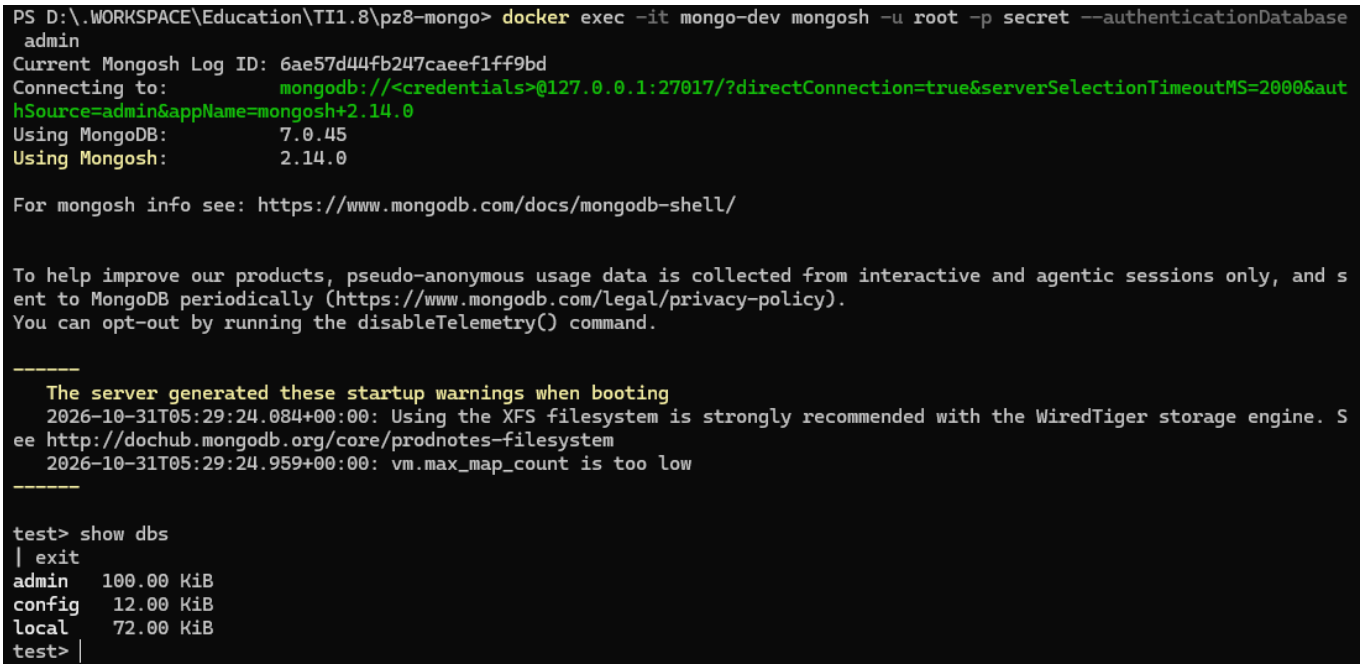


Рисунок 7 — Подключение к MongoDB через `mongosh` и выполнение `show dbs`.

5. Реализация приложения на Go

5.1. Подключение к MongoDB

В файле `internal/db/mongo.go` реализована функция `ConnectMongo`. Она принимает контекст, URI подключения и имя базы данных. Для подключения создаётся контекст с таймаутом 10 секунд. После установления соединения выполняется `Ping` с таймаутом 3 секунды.

```
package db

import (
    "context"
    "time"

    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
)

type MongoDeps struct {
    Client   *mongo.Client
    Database *mongo.Database
}

func ConnectMongo(ctx context.Context, uri, dbName string) (*MongoDeps, error)
{
    dialCtx, cancel := context.WithTimeout(ctx, 10*time.Second)
```

```
    defer cancel()

    client, err := mongo.Connect(dialCtx, options.Client().ApplyURI(uri))
    if err != nil {
        return nil, err
    }

    pingCtx, cancelPing := context.WithTimeout(ctx, 3*time.Second)
    defer cancelPing()
    if err := client.Ping(pingCtx, nil); err != nil {
        _ = client.Disconnect(context.Background())
        return nil, err
    }

    return &MongoDeps{Client: client, Database: client.Database(dbName)}, nil
}
```

Структура `MongoDeps` объединяет клиент MongoDB и объект выбранной базы данных. Если проверка соединения завершается ошибкой, клиент закрывается, а ошибка возвращается вызывающему коду. Это позволяет обнаружить проблему подключения при старте приложения, а не во время первого пользовательского запроса.

5.2. Модель документа

В `internal/notes/model.go` описывается структура `Note`:

```
package notes

import (
    "time"
    "go.mongodb.org/mongo-driver/bson/primitive"
)

type Note struct {
    ID        primitive.ObjectID `bson:"_id,omitempty" json:"id"`
    Title     string             `bson:"title" json:"title"`
    Content   string             `bson:"content" json:"content"`
    CreatedAt time.Time          `bson:"createdAt" json:"createdAt"`
    UpdatedAt time.Time          `bson:"updatedAt" json:"updatedAt"`
}
```

Теги `bson` задают имена полей при сохранении документа в MongoDB, а теги `json` — имена полей в HTTP JSON. Идентификатор имеет тип `primitive.ObjectID`. Поле `_id`

помечено `omitempty`, поэтому при первоначальной вставке его можно не задавать: MongoDB сформирует идентификатор автоматически.

Дополнительно в модели определена структура `Stats`, используемая бонусным методом агрегирования статистики.

5.3. Репозиторий и создание коллекции с индексом

Файл `internal/notes/repo.go` содержит методы доступа к коллекции `notes`. При создании репозитория вызывается `database.Collection("notes")`, после чего создаётся уникальный индекс по полю `title`:

```
_, err := collection.Indexes().CreateOne(context.Background(),
mongo.IndexModel{
    Keys:    bson.D{{Key: "title", Value: 1}},
    Options: options.Index().SetUnique(true).SetName("uniq_note_title"),
})
```

Значение `1` задаёт сортировку индекса по возрастанию. Параметр `SetUnique(true)` запрещает повторяющиеся значения поля `title`. Если в коллекции уже есть документы с одинаковыми заголовками, создание уникального индекса завершится ошибкой — в таком случае дубликаты необходимо устранить.

Репозиторий реализует следующие методы:

- `Create` — формирует заметку с датами создания и изменения, вставляет её через `InsertOne` и возвращает присвоенный MongoDB ObjectID;
- `ByID` — преобразует строковый ID в ObjectID и ищет документ по `_id`;
- `List` — возвращает список заметок, поддерживает поиск по заголовку через регулярное выражение без учёта регистра, сортировку и пагинацию;
- `Update` — применяет `$set` только к переданным полям и обновляет `updatedAt`;
- `Delete` — удаляет документ по ObjectID и проверяет количество удалённых записей;
- `GetStats` — выполняет aggregation pipeline и вычисляет количество документов и среднюю длину содержимого.

Для обновления используется `FindOneAndUpdate` с `SetReturnDocument(options.After)`, поэтому метод возвращает уже обновлённый документ. Если документ не найден, возвращается собственная ошибка `ErrNotFound`.

5.4. HTTP-обработчики и маршрутизация

В файле `internal/notes/handler.go` реализованы HTTP-обработчики. Для маршрутизации используется библиотека `chi`.

Маршруты коллекции:

HTTP-метод	Маршрут	Назначение	Успешный код
POST	`/api/v1/notes`	Создание заметки	201 Created
GET	`/api/v1/notes`	Получение списка, поиск, пагинация	200 OK
GET	`/api/v1/notes/{id}`	Получение заметки по ID	200 OK
PATCH	`/api/v1/notes/{id}`	Частичное обновление	200 OK
DELETE	`/api/v1/notes/{id}`	Удаление заметки	204 No Content
GET	`/api/v1/notes/stats`	Дополнительная статистика	200 OK

Для каждого запроса к репозиторию создаётся контекст с таймаутом 5 секунд. Это ограничивает время ожидания операций базы данных.

При создании заметки обработчик проверяет корректность JSON и наличие заголовка. При дублировании заголовка возвращается HTTP 409. Для некорректного JSON используется HTTP 400. Если заметка не найдена или передан некорректный ObjectID, возвращается HTTP 404.

Для `PATCH` используются указатели `*string`. Благодаря этому можно отличить поле, которое отсутствует в JSON, от поля, переданного со значением пустой строки. Обновляются только присутствующие поля.

5.5. Точка входа приложения

В `cmd/api/main.go` находится функция `main`. При запуске приложение:

1. Загружает настройки из `.env` (если файл существует).
2. Получает URI MongoDB, имя базы и адрес HTTP-сервера.
3. Подключается к MongoDB и проверяет соединение.
4. Создаёт репозиторий и уникальный индекс.
5. Регистрирует маршрут `/health` и маршруты заметок.
6. Запускает HTTP-сервер на порту 8080.
7. При завершении закрывает HTTP-сервер и соединение с MongoDB.

Маршрут `/health` возвращает JSON `{"status":"ok"}` и позволяет отдельно проверить работу HTTP-сервера.

6. Запуск приложения

Перед запуском необходимо убедиться, что контейнер MongoDB работает:

```
docker compose ps
```

Затем в корне проекта выполняются команды:

```
go mod tidy
go run ./cmd/api
```

`go mod tidy` проверяет импорты и приводит зависимости Go-модуля в согласованное состояние. `go run ./cmd/api` собирает и запускает HTTP-сервис.

При успешном запуске в терминале появится сообщение о прослушивании адреса `:8080`. Терминал с сервером следует оставить открытым. Все дальнейшие запросы выполняются во втором окне PowerShell.

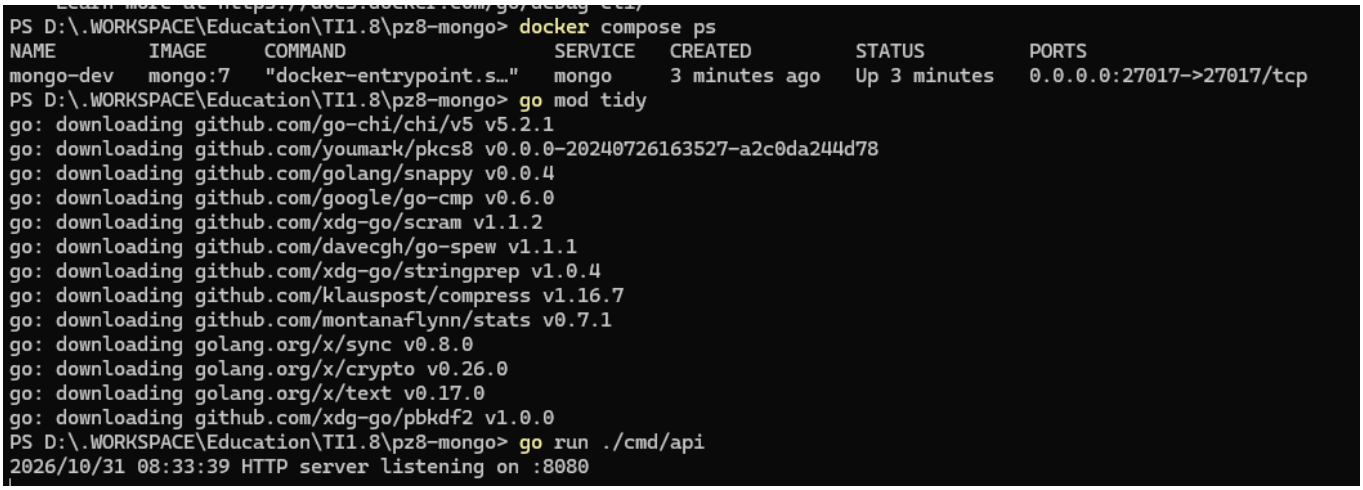


Рисунок 8 — Успешный запуск Go-сервера и сообщение о прослушивании порта 8080.

Проверим состояние сервера:

```
Invoke-RestMethod -Uri 'http://localhost:8080/health'
```

Ожидаемый результат:

```
status
------
```

```
ok
```

```
PS C:\Users\Falcion> Invoke-RestMethod -Uri 'http://localhost:8080/health'

status
------
ok


PS C:\Users\Falcion>
```

Рисунок 9 — Успешный ответ endpoint `/health`.

7. Проверка CRUD-операций

Все примеры ниже предназначены для PowerShell. Для удобства результат создания сохраняется в переменную `$created`, а идентификатор — в `$id`. Запросы выполняются последовательно в одном окне PowerShell.

7.1. Create — создание заметки

Выполним POST-запрос:

```
$body = @{
    title = 'First note'
    content = 'Hello Mongo!'
} | ConvertTo-Json

$created = Invoke-RestMethod `
    -Method Post `
    -Uri 'http://localhost:8080/api/v1/notes' `
    -ContentType 'application/json' `
    -Body $body

$created
$id = $created.id
```

Сервис должен вернуть созданный документ с полями `id`, `title`, `content`, `createdAt` и `updatedAt`. Идентификатор создаётся MongoDB автоматически. HTTP-статус успешного создания — `201 Created`.

```
PS C:\Users\Falcion> $body = @{
>>     title = 'First note'
>>     content = 'Hello Mongo!'
>> } | ConvertTo-Json
PS C:\Users\Falcion>
PS C:\Users\Falcion> $created = Invoke-RestMethod `
>>     -Method Post `
>>     -Uri 'http://localhost:8080/api/v1/notes' `
>>     -ContentType 'application/json' `
>>     -Body $body
PS C:\Users\Falcion>
PS C:\Users\Falcion> $created


id        : 6ae57dd9f200cb317e0156e8
title     : First note
content   : Hello Mongo!
createdAt : 2026-10-31T05:34:17.6730728Z
updatedAt : 2026-10-31T05:34:17.6730728Z
```

Рисунок 10 — Создание заметки методом POST и полученный ObjectID.

7.2. Read — получение списка заметок

Получим первые пять заметок:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?limit=5&skip=0'
```

Параметр `limit` задаёт максимальное количество документов, а `skip` — количество документов, которые нужно пропустить. По умолчанию сервис возвращает не более 20 записей, а максимально допустимое значение `limit` ограничено 200.

Проверим поиск по заголовку:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?
limit=5&skip=0&q=first'
```

Параметр `q` используется как условие поиска по полю `title`. Поиск выполняется без учёта регистра символов.

```
PS C:\Users\Falcion> Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?limit=5&skip=0'


id        : 6ae57dd9f200cb317e0156e8
title     : First note
content   : Hello Mongo!
createdAt : 2026-10-31T05:34:17.673Z
updatedAt : 2026-10-31T05:34:17.673Z



PS C:\Users\Falcion> Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes?limit=5&skip=0&q=first'


id        : 6ae57dd9f200cb317e0156e8
title     : First note
content   : Hello Mongo!
createdAt : 2026-10-31T05:34:17.673Z
updatedAt : 2026-10-31T05:34:17.673Z
```

Рисунок 11 — Получение списка заметок с параметрами пагинации и поиском.

7.3. Read — получение заметки по идентификатору

Используем ID, полученный при создании:

```
Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"
```

Если документ существует, API вернёт его содержимое. Если ID некорректен или документ отсутствует, сервис вернёт `404 Not Found`.

```
PS C:\Users\Falcion> Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"


id        : 6ae57dd9f200cb317e0156e8
title     : First note
content   : Hello Mongo!
createdAt : 2026-10-31T05:34:17.673Z
updatedAt : 2026-10-31T05:34:17.673Z
```

Рисунок 12 — Получение одной заметки по ObjectID.

7.4. Update — частичное обновление заметки

Изменим только содержимое, не передавая заголовок:

```
$patch = @{
    content = 'Updated content'
} | ConvertTo-Json

$updated = Invoke-RestMethod `
    -Method Patch `
    -Uri "http://localhost:8080/api/v1/notes/$id" `
    -ContentType 'application/json' `
```

```
    -Body $patch

$updated
```

Операция PATCH использует оператор MongoDB `$set`. Так как в JSON передано только поле `content`, заголовок остаётся прежним. Поле `updatedAt` получает новое значение времени.

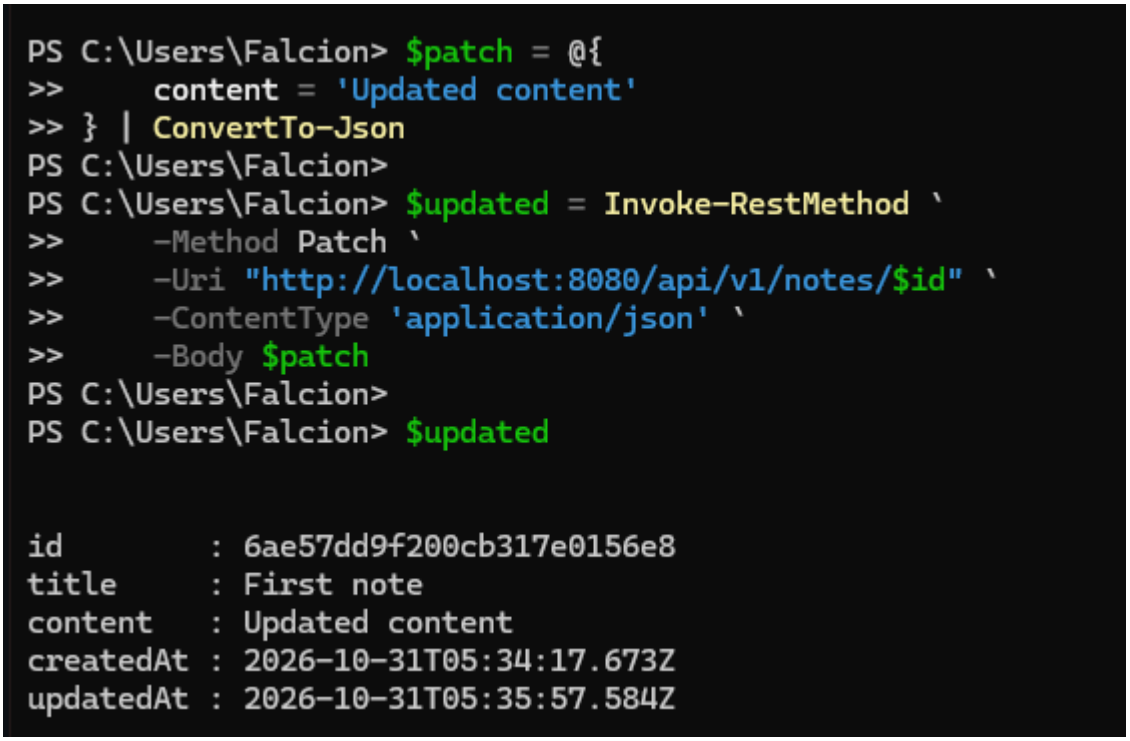


Рисунок 13 — Частичное обновление заметки методом PATCH.

7.5. Delete — удаление заметки

Удалим созданный документ:

```
Invoke-WebRequest `
    -Method Delete `
    -Uri "http://localhost:8080/api/v1/notes/$id"
```

При успешном удалении сервер возвращает HTTP `204 No Content`. Это означает, что операция выполнена, но тело ответа отсутствует.

Для проверки повторно запросим удалённую заметку:

```
try {
    Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"
} catch {
```

```
    $_.Exception.Response.StatusCode.value__
}
```

Ожидаемый код — `404`, поскольку документ больше не существует.

```
PS C:\Users\Falcion> Invoke-WebRequest `
>>     -Method Delete `
>>     -Uri "http://localhost:8080/api/v1/notes/$id"


StatusCode        : 204
StatusDescription : No Content
Content           : {}
RawContent        : HTTP/1.1 204 No Content
                    Date: Sat, 31 Oct 2026 05:36:12 GMT


Headers           : {[Date, Sat, 31 Oct 2026 05:36:12 GMT]}
RawContentLength  : 0



PS C:\Users\Falcion> try {
>>     Invoke-RestMethod -Uri "http://localhost:8080/api/v1/notes/$id"
>> } catch {
>>     $_.Exception.Response.StatusCode.value__
>> }
404
```

Рисунок 14 — Удаление заметки и проверка отсутствия удалённого документа.

8. Проверка уникального индекса и обработки ошибок

Попробуем создать две заметки с одинаковым заголовком. Первый запрос:

```
$duplicateBody = @{ title = 'Unique title test'; content = 'First' } |
ConvertTo-Json
Invoke-RestMethod -Method Post -Uri 'http://localhost:8080/api/v1/notes' -
ContentType 'application/json' -Body $duplicateBody
```

Второй запрос с тем же заголовком:

```
$duplicateBody2 = @{ title = 'Unique title test'; content = 'Second' } |
ConvertTo-Json
try {
    Invoke-RestMethod -Method Post -Uri 'http://localhost:8080/api/v1/notes' -
```

```
ContentType 'application/json' -Body $duplicateBody2
} catch {
    $_.Exception.Response.StatusCode.value__
}
```

Ожидаемый результат второго запроса — HTTP 409. Он подтверждает, что уникальный индекс запрещает повторяющиеся заголовки, а приложение корректно обрабатывает ошибку MongoDB.



Рисунок 15 — Проверка ограничения уникальности: повторная вставка заголовка приводит к HTTP 409.

9. Дополнительное задание: aggregation pipeline

В качестве бонусной возможности реализован endpoint статистики:

```
Invoke-RestMethod -Uri 'http://localhost:8080/api/v1/notes/stats'
```

В репозитории используется aggregation pipeline MongoDB. На этапе `$group` вычисляются:

- `count` — количество документов;
- `averageContentLength` — средняя длина содержимого заметок, вычисляемая через `$avg` и `$strLenCP`.

Пример формы ответа:

```
{
  "count": 2,
```

```
  "averageContentLength": 18.5
}
```

Значения в примере приведены для иллюстрации формата. Фактические числа зависят от документов, находящихся в коллекции в момент запроса. Если коллекция пуста, endpoint возвращает нулевые значения.

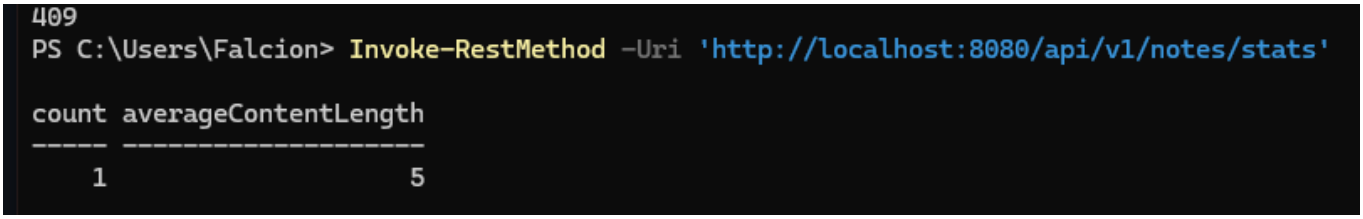


Рисунок 16 — Получение статистики заметок с помощью aggregation pipeline.

10. Автоматизированный тест

В проект добавлен интеграционный тест `internal/notes/repo_test.go`. Он проверяет последовательность создания заметки, чтения по ObjectID, частичного обновления и удаления.

Тест использует отдельную базу данных с именем, содержащим текущую дату и время. После завершения теста база удаляется. Это позволяет не затрагивать основную учебную базу `pz8`.

В PowerShell зададим URI подключения для текущего сеанса и запустим тесты:

```
$env:MONGO_URI = 'mongodb://root:secret@localhost:27017/?authSource=admin'
go test ./...
```

Если переменная `MONGO_URI` не задана, интеграционный тест пропускается, чтобы обычная проверка пакетов не требовала запущенной MongoDB. При работающем контейнере тест должен завершиться успешно.

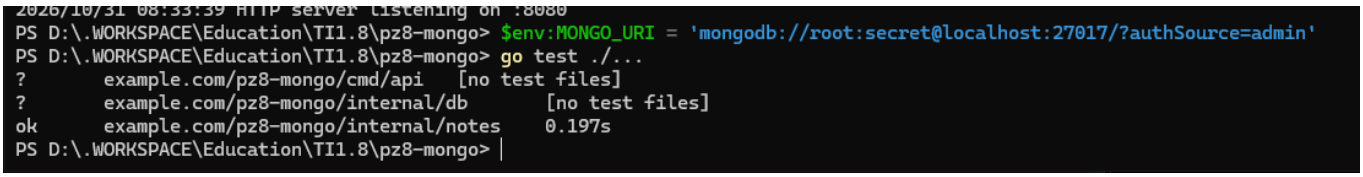


Рисунок 17 — Результат выполнения `go test ./...`.

11. Итоговая структура проекта

После выполнения работы структура должна иметь следующий вид:

```
pz8-mongo/
├── cmd/
│   └── api/
│       └── main.go
├── internal/
│   ├── db/
│   │   └── mongo.go
│   └── notes/
│       ├── model.go
│       ├── repo.go
│       ├── handler.go
│       └── repo_test.go
├── .env.example
├── .env
├── .gitignore
├── docker-compose.yml
├── go.mod
├── go.sum
└── README.md
```

Файл `.env` является локальным и исключён из Git через `.gitignore`. Файл `go.sum` формируется автоматически после загрузки зависимостей командой `go mod tidy` или `go get`.

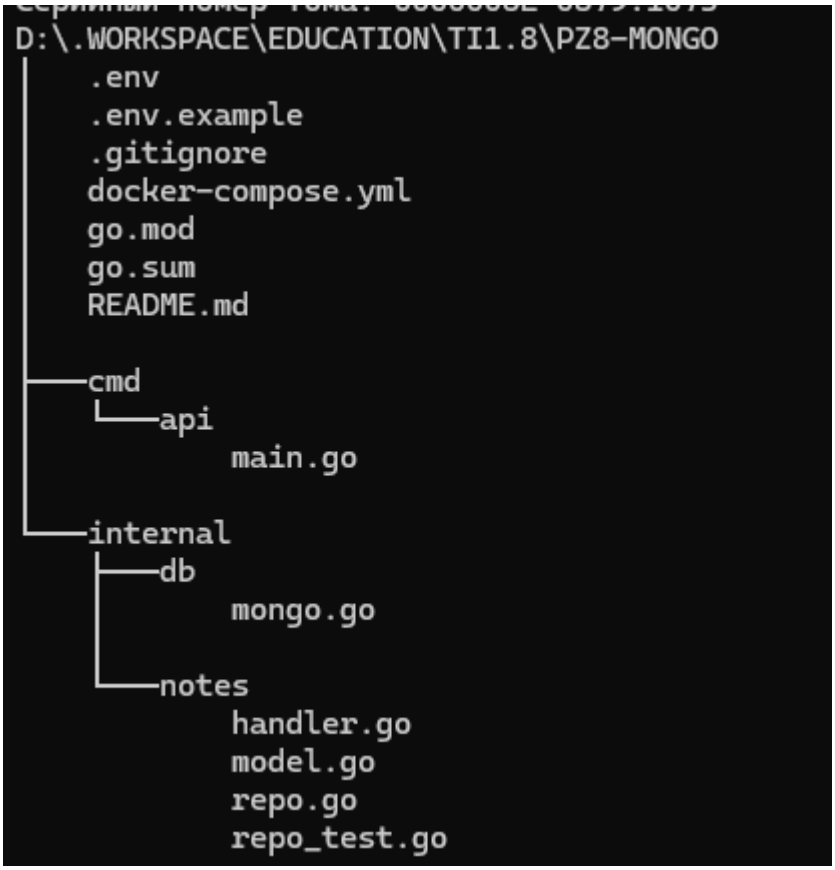


Рисунок 18 — Итоговая структура проекта в редакторе кода.

12. Остановка приложения и контейнера

Сначала остановим Go-сервер сочетанием клавиш `Ctrl+C` в терминале, где он запущен. Затем в PowerShell из каталога проекта выполним:

```
docker compose down
```

Команда останавливает и удаляет контейнер, но сохраняет именованный том с данными MongoDB. Если требуется полностью удалить и данные, применяется команда:

```
docker compose down -v
```

В рамках обычного завершения практической работы рекомендуется использовать вариант без `-v`, чтобы не удалять сохранённые данные.

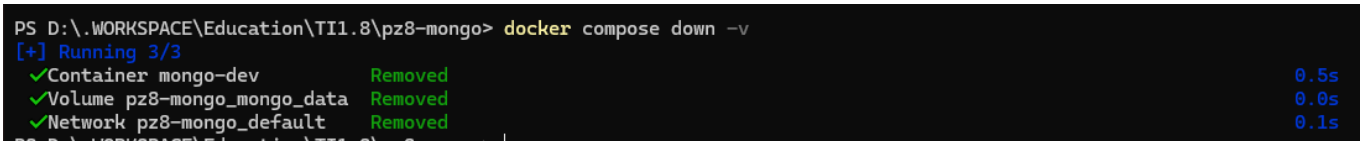


Рисунок 19 — Остановка контейнера MongoDB командой `docker compose down`.

13. Ответы на контрольные вопросы

1. Чем документная модель MongoDB принципиально отличается от реляционной? Когда она удобнее?

В реляционной модели данные представлены таблицами с заранее определёнными столбцами и связями между таблицами. В MongoDB данные хранятся в виде BSON-документов внутри коллекций. Документы одной коллекции могут иметь различный набор полей и содержать вложенные объекты или массивы. Документная модель удобна, когда структура данных часто меняется, данные естественным образом представлены вложенными объектами или требуется хранить JSON-подобные структуры, например профили, заметки и каталоги товаров. Реляционные СУБД обычно предпочтительнее там, где важны сложные связи, строгие ограничения целостности и транзакционные сценарии.

2. Что такое ObjectID и зачем нужен `_id`? Как корректно парсить/валидировать его в Go?

`_id` — уникальный идентификатор документа в коллекции. Если приложение не задаёт его самостоятельно, MongoDB генерирует его автоматически. `ObjectID` — специальный тип идентификатора MongoDB, содержащий временную составляющую, случайное значение и счётчик. В Go он представлен типом `primitive.ObjectID`. Строковый идентификатор преобразуется функцией `primitive.ObjectIDFromHex(idHex)`. Если преобразование завершается ошибкой, переданный идентификатор некорректен; в реализации такой случай отображается в `404 Not Found`.

3. Какие операции CRUD предоставляет драйвер MongoDB и какие операторы обновления вы знаете?

Для создания применяются `InsertOne` и `InsertMany`; для чтения — `FindOne` и `Find`; для обновления — `UpdateOne`, `UpdateMany` и `FindOneAndUpdate`; для удаления — `DeleteOne` и `DeleteMany`. Среди операторов обновления: `$set` задаёт значения полей, `$inc` увеличивает числовое значение, `$push` добавляет элемент в массив, `$pull` удаляет из массива элементы, соответствующие условию. В этой работе для частичного изменения заметки используется `$set`.

4. Как устроены индексы в MongoDB? Как создать уникальный индекс и чем он грозит при вставке?

Индекс — отдельная структура данных, позволяющая быстрее находить документы по значениям выбранных полей. MongoDB поддерживает одиночные, составные, уникальные и TTL-индексы. Уникальный индекс можно создать через `collection.Indexes().CreateOne(...)`, передав `mongo.IndexModel`, ключ поля и опцию `SetUnique(true)`. В данной работе индекс создаётся по `title`. Если приложение

попытается вставить документ с уже существующим значением этого поля, MongoDB отклонит операцию с ошибкой дублирования ключа. API преобразует её в HTTP 409 Conflict.

5. Почему важно использовать `context.WithTimeout` при вызовах к базе? Что произойдёт при его срабатывании?

`context.WithTimeout` ограничивает максимальное время выполнения операции. Без ограничения запрос может слишком долго ожидать ответа при сетевой проблеме или недоступности сервера. По истечении таймаута контекст отменяется, а операция MongoDB завершается с ошибкой, связанной с отменой или истечением срока действия контекста. Это позволяет приложению освобождать ресурсы и возвращать управление, а не удерживать обработчик бесконечно.

14. Вывод

В ходе практической работы были рассмотрены основные понятия документной СУБД MongoDB: база данных, коллекция, BSON-документ и ObjectID. Было подготовлено окружение на основе Docker Compose, создан Go-модуль и реализовано подключение к MongoDB с проверкой соединения через `Ping` и использованием контекстов с таймаутами.

Разработан REST-сервис для управления заметками. В коллекции `notes` используется уникальный индекс по заголовку. Реализованы создание, получение списка, поиск по заголовку, пагинация, получение документа по ObjectID, частичное обновление и удаление. Также реализована обработка типовых ошибок: некорректных идентификаторов, отсутствующих документов и дублирования уникального поля.

Дополнительно реализован aggregation pipeline для расчёта статистики заметок и интеграционный тест репозитория. Таким образом, получено приложение на Go, взаимодействующее с MongoDB и предоставляющее HTTP API для выполнения основных CRUD-операций.